K.R. Mangalam University

School of Engineering and Technology

LAB 3 REPORT

Smart Resource Allocation and Scheduling System using Greedy Algorithms

Program: BCA (AI & DS) Course Code - ENCA351 (2026-27)

Submitted by:

Name: Bhavya Rattan
RollNumber: 2401201004
Course: BCA (AI & DS)
Section: B

Submitted To:

Dr. Aarti Sangwan
Assistant Professor

1. Introduction

A greedy algorithm builds a solution step by step and, at every step, makes the choice that looks best at that moment without reconsidering it later. When a problem has the *greedy-choice property* and *optimal substructure*, this simple strategy gives a globally optimal answer very efficiently. This lab applies the greedy method to three practical problems, implements them in Python, visualizes their behaviour, and then benchmarks them to compare measured performance with theoretical complexity.

Objectives

- **Resource allocation** - Fractional Knapsack: maximize the value that fits in a limited capacity.
- **Task scheduling** - Activity Selection: choose the maximum number of non-overlapping activities.
- **Data compression** - Huffman Coding: build optimal prefix-free binary codes.
- **Benchmarking (Task 1)** - record execution time, memory usage and operation counts for nine input sizes and prepare a theoretical vs observed complexity table.
- **Visualization (Task 2)** - generate visualizations and comparative charts with performance observations.

2. Environment and Tools

Item	Details
Language	Python 3 (CPython)
Algorithms	algorithms/ package: fractional_knapsack.py, activity_selection.py, huffman_coding.py
Timing	time.perf_counter() read immediately before and after each call; one run per (algorithm, n) on a freshly generated input
Memory	tracemalloc peak traced allocation (KB), started just before the algorithm call, so the input itself is not counted
Analysis and charts	pandas, NumPy (log-log regression), matplotlib
Interfaces	Benchmark script (analysis/generate_benchmarks_and_charts.py), menu-driven app (app.py) and Jupyter notebook, all using the same modules
Reproducibility	random.seed(42); results are exported to data_csv/benchmark_results.csv and charts to the graphes/ folder

Source repository: github.com/bhavyarattan21/DAA_Assignment-3_Bhavya_Rattan_2401201004

3. Methodology and Test Configuration

For each input size n the benchmark (1) builds a fresh random input for every algorithm, (2) starts tracemalloc, (3) times the algorithm call with time.perf_counter(), (4) reads the peak traced memory and stops tracing, and (5) appends one record to a pandas DataFrame that is exported to CSV. Operation counts are key comparisons made by the sort (Knapsack: value/weight ratio, Activity: finish time) or by the min-heap (Huffman), plus the iterations of the main greedy loop (Huffman also counts the frequency scan, the n-1 heap merges and the n code look-ups). They were obtained from an instrumented run with the same generators and seed. The growth exponent k in $T \sim n^k$ is estimated as the slope of a least-squares line through (log n, log T) over all nine sizes.

Algorithm	Input sizes tested (n)	Input type	Greedy criterion
Fractional Knapsack	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	n random items, weight in [5,100], value in [10,500]; capacity = 25n	Highest value-to-weight ratio
Activity Selection	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	n random activities, start in [0,2n], duration 1-50	Earliest finish time first

Algorithm	Input sizes tested (n)	Input type	Greedy criterion
Huffman Coding	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	Text of n characters from 62 letters/digits, skewed frequencies (weight 1/(i+1))	Merge two lowest-frequency nodes

Table 1: Test configuration (3 algorithms x 9 input sizes = 27 measurements).

4. Algorithms Implemented

Group	Algorithm	Greedy rule	Time / Space
Resource allocation	Fractional Knapsack	Take items in decreasing value-to-weight ratio; split the last item if needed	O(n log n) / O(n)
Task scheduling	Activity Selection	Sort by finish time; pick an activity if it starts at or after the last selected finish	O(n log n) / O(n)
Data compression	Huffman Coding	Repeatedly merge the two lowest-frequency nodes with a min-heap	O(n log n) / O(n)*

* For Huffman Coding, n is the number of distinct characters. The complete source code is in Appendix A.

5. Visualizations and Performance Observations

5.1 Resource allocation: Fractional Knapsack

Items: Gold (10 kg, 60), Silver (20 kg, 100), Bronze (30 kg, 120), Platinum (15 kg, 105), Copper (25 kg, 50); capacity W = 50 kg. The greedy rule sorts by value-to-weight ratio (Platinum 7.0, Gold 6.0, Silver 5.0, Bronze 4.0, Copper 2.0).

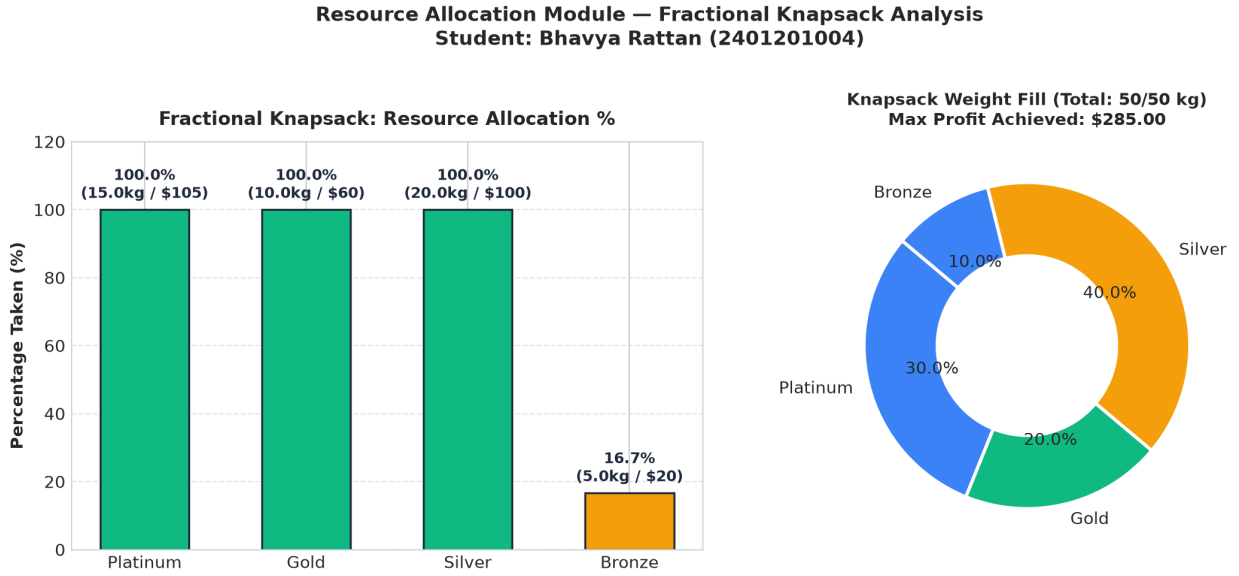


Figure 1: Fractional knapsack allocation: percentage of each item taken (left) and share of the 50 kg capacity (right).

- **Highest ratio first.** Platinum, Gold and Silver are taken whole (15 + 10 + 20 = 45 kg, value 105 + 60 + 100 = 265).
- **One fractional item.** The remaining 5 kg takes 16.7% of Bronze (5/30), adding value 20. Total maximum profit = **285.00**, with the knapsack exactly full (50/50 kg). Copper, the lowest ratio, is never taken.
- **Why it is optimal.** Because any fraction is allowed, swapping weight from a lower-ratio item to a higher-ratio one can never reduce value (exchange argument), so the greedy choice is globally optimal. The same rule is not optimal for the 0/1 knapsack.

5.2 Task scheduling: Activity Selection

Eight candidate activities (A1 [1,4], A2 [3,5], A3 [0,6], A4 [5,7], A5 [3,9], A6 [5,9], A7 [6,10], A8 [8,11]) are sorted by earliest finish time; an activity is selected only if its start is at or after the finish of the last selected one.

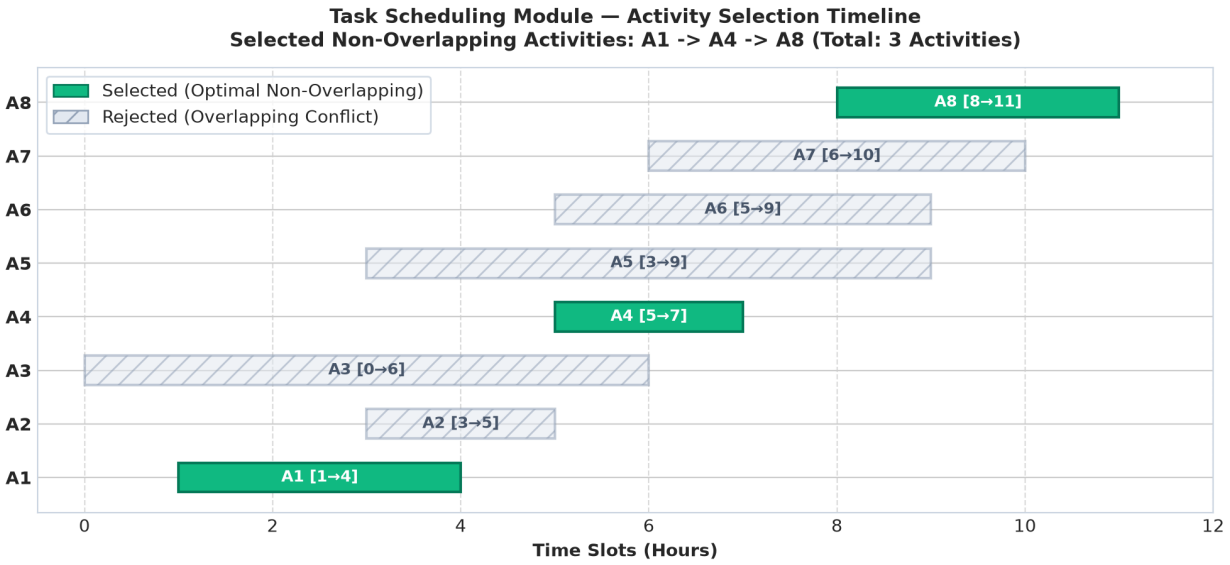


Figure 2: Activity selection timeline: selected (solid) and rejected (hatched) activities.

- **Result.** The greedy schedule is A1 [1-4], A4 [5-7], A8 [8-11]: **3 of 8** activities, with no overlaps.
- **Why the rest are rejected.** A2, A3 and A5 start before A1 finishes (time 4); A6 starts at 5, before A4 finishes (7); A7 starts at 6, also before A4 finishes. Finishing early leaves the most time free for later activities.
- **Cost.** Sorting dominates, so time is O(n log n); the single scan afterwards is O(n). Extra space is O(n) for the sorted copy and the result list.
- **Metric matters.** Shortest-duration-first fails (for [0,5], [4,6], [5,10] it picks only [4,6]); earliest-finish-first is provably optimal.

5.3 Data compression: Huffman Coding

Input string: **abracadabra** (11 characters, 5 distinct). The two lowest-frequency nodes are merged repeatedly with a min-heap.

Bhavya Rattan (Roll No: 2401201004) | DAA Lab 3

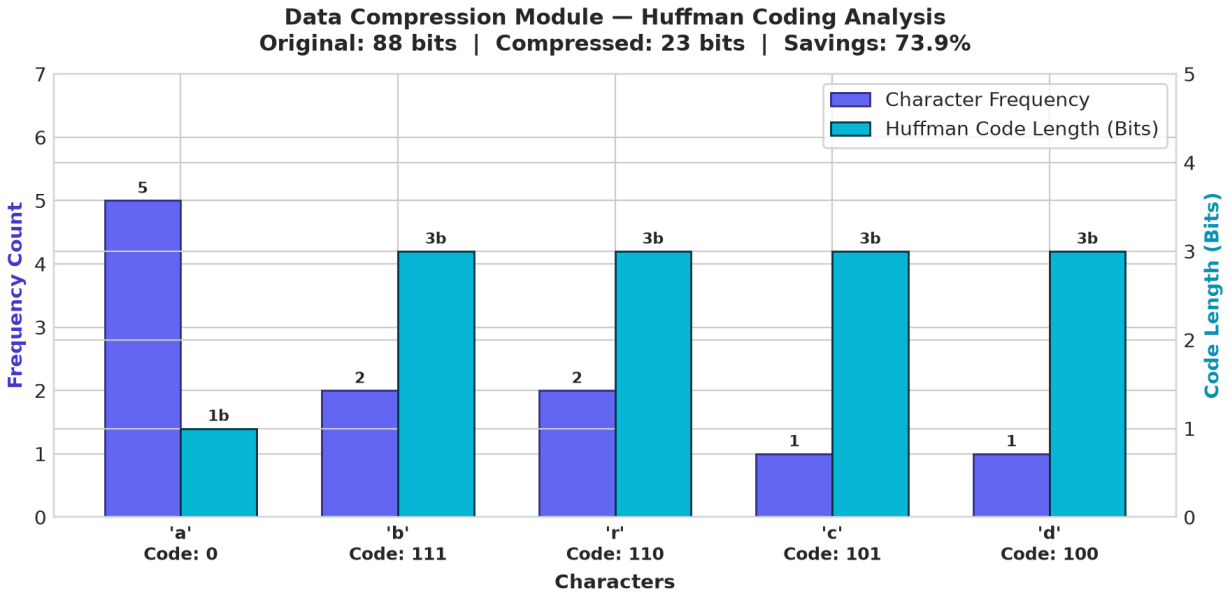


Figure 3: Character frequency versus assigned Huffman code length.

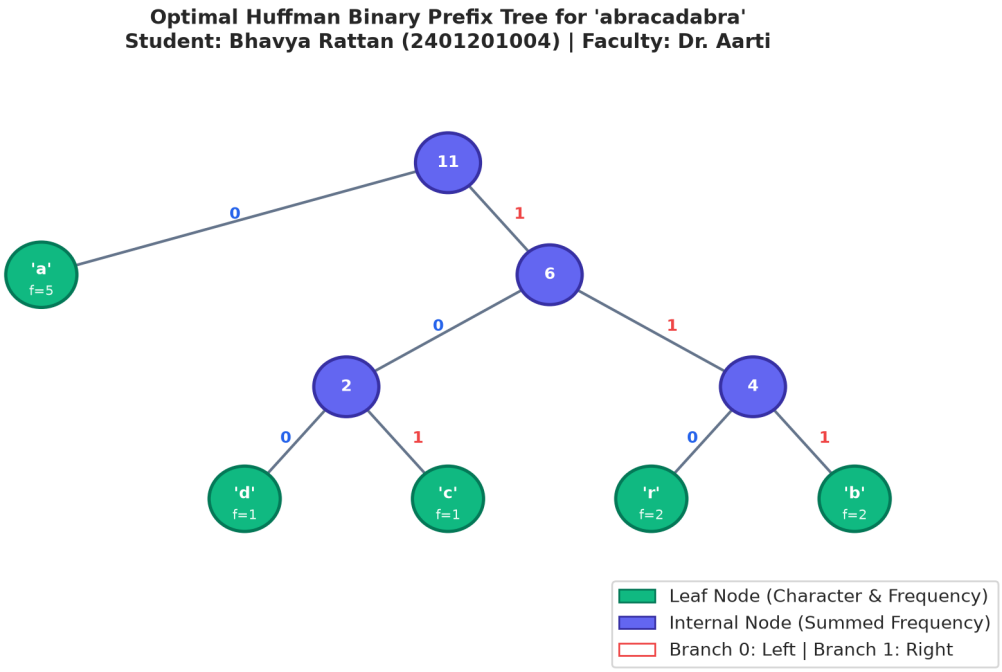


Figure 4: Huffman prefix tree for 'abracadabra' (left branch = 0, right branch = 1).

- **Short codes for frequent symbols.** 'a' (5 occurrences) gets the 1-bit code 0; 'b', 'r', 'c', 'd' get 3-bit codes (111, 110, 101, 100).
- **Compression.** Encoded length = 5x1 + 2x3 + 2x3 + 1x3 + 1x3 = **23 bits** against 88 bits in 8-bit ASCII (**73.9% saving**, average 2.09 bits per character). A fixed 3-bit code would need 33 bits, so the variable-length code is also 30% smaller than the best fixed-length code.
- **Prefix-free.** No codeword is a prefix of another, so the bit stream decodes unambiguously with a single left-to-right walk of the tree.

6. Empirical Benchmarking (Task 1)

Each algorithm was run on random inputs of size n = 10 to 10,000 (seed 42). Time was measured with time.perf_counter() and peak memory with tracemalloc. Raw data: data_csv/benchmark_results.csv.

6.1 Sample output

```
$ python analysis/generate_benchmarks_and_charts.py
[*] Running comprehensive benchmark experiments for all 3 greedy algorithms...
  Benchmarking size n = 10 ... (continues for 50, 100, 250, 500, 1000, 2500, 5000)
  Benchmarking size n = 10000 ...
[OK] Benchmark results saved to data_csv/benchmark_results.csv

      n | Knap ms   Knap KB | Act ms    Act KB | Huff ms  Huff KB
-----------------------------------------------------------------
     10 |  0.1191      2.75 |  0.0859     2.48 |  0.3043     2.92
     50 |  0.3578      7.41 |  0.3858    10.68 |  0.5298     8.00
    100 |  0.6755     14.35 |  0.7214    20.27 |  0.7476    10.97
    250 |  2.4506     41.91 |  1.9168    54.14 |  1.1844    19.46
    500 |  3.7750     82.68 |  4.0090   110.04 |  1.4236    23.88
  1,000 |  8.0878    166.93 |  8.2191   223.14 |  1.7599    31.75
  2,500 | 15.2443    431.63 | 13.9410   564.02 |  1.8480    50.90
  5,000 | 48.2021    836.10 | 36.2427  1126.12 |  2.9401    84.16
 10,000 |140.2810   1923.62 | 60.1385  2261.45 |  5.9603   151.36
```

Figure 5: Console output of the benchmark script and the exported benchmark_results.csv.

6.2 Measured execution time and peak memory

n	Knapsack time (ms)	Knapsack mem (KB)	Activity time (ms)	Activity mem (KB)	Huffman time (ms)	Huffman mem (KB)
10	0.119	2.8	0.086	2.5	0.304	2.9
50	0.358	7.4	0.386	10.7	0.530	8.0
100	0.675	14.4	0.721	20.3	0.748	11.0
250	2.451	41.9	1.917	54.1	1.184	19.5
500	3.775	82.7	4.009	110.0	1.424	23.9
1,000	8.088	166.9	8.219	223.1	1.760	31.7
2,500	15.244	431.6	13.941	564.0	1.848	50.9
5,000	48.202	836.1	36.243	1,126.1	2.940	84.2
10,000	140.281	1,923.6	60.139	2,261.5	5.960	151.4

Table 2: Measured execution time and peak memory (all 27 measurements).

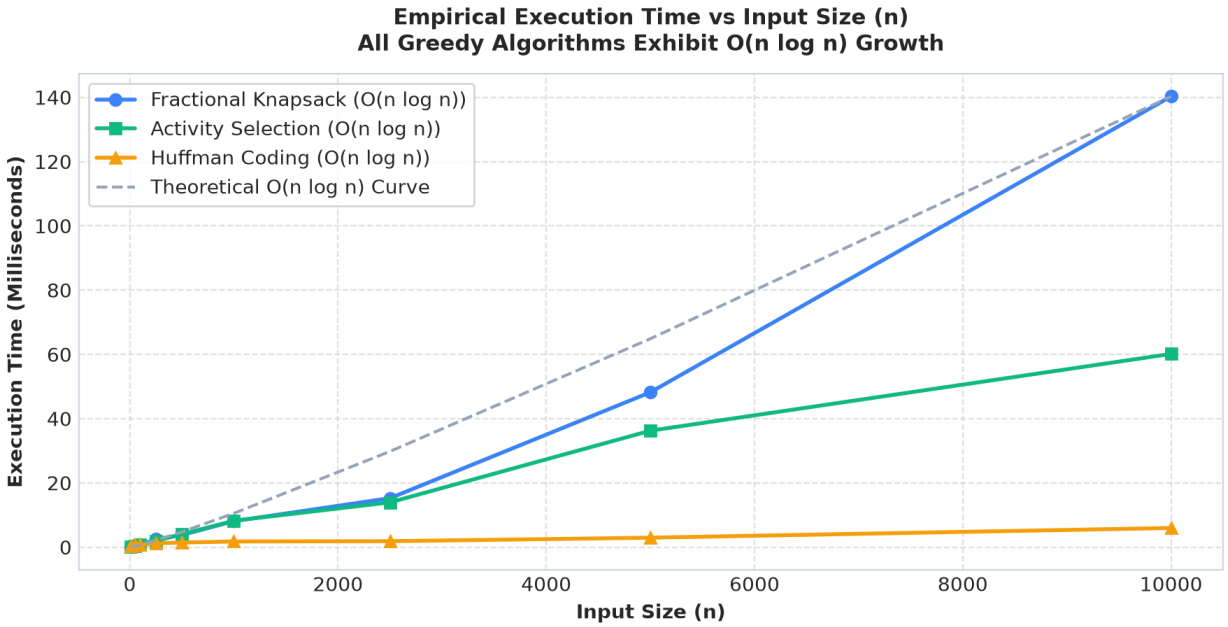


Figure 6: Execution time versus input size, with a theoretical O(n log n) reference scaled to the knapsack run at n = 10,000.

Bhavya Rattan (Roll No: 2401201004) | DAA Lab 3 Benchmarks

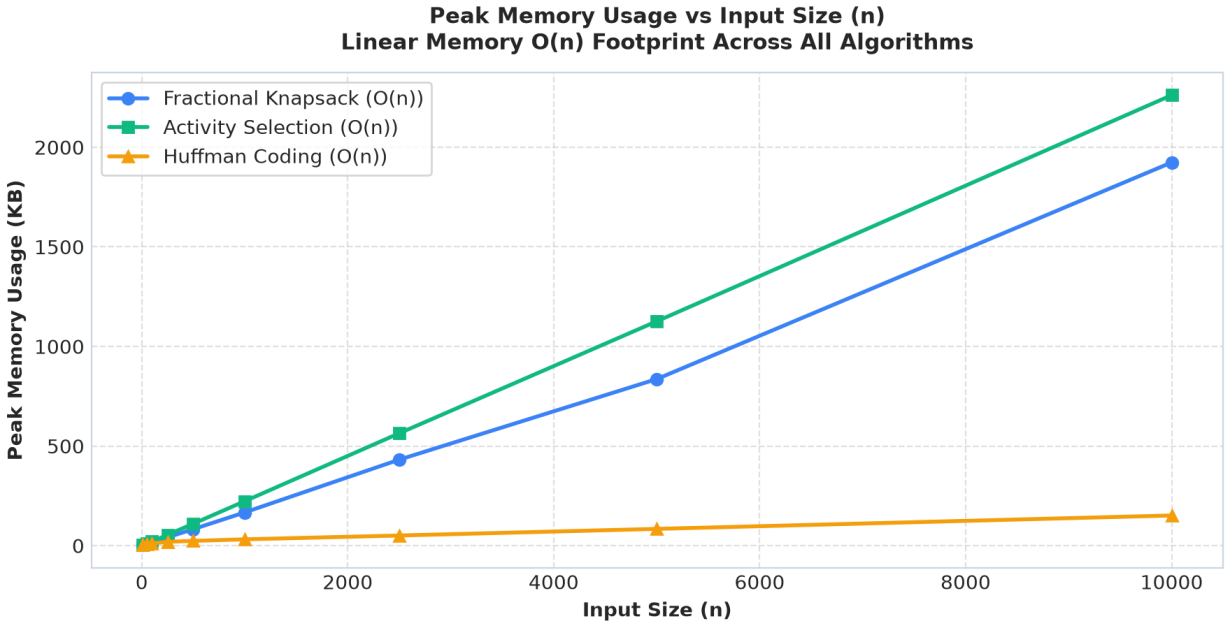


Figure 7: Peak memory versus input size.

- **Knapsack and activity selection grow like n log n.** Fitted log-log time slopes are 1.01 (knapsack) and 0.96 (activity selection). From n = 100 to n = 10,000 (100x) knapsack time rose 208x and activity selection 83x; a pure n log n law predicts about 200x.
- **Memory is linear for both.** Fitted memory slopes are 0.98 and 1.00. At n = 10,000 knapsack peaked at 1,923.6 KB and activity selection at 2,261.5 KB, because each builds O(n) helper objects and a sorted copy.
- **Huffman grows much more slowly in n.** Time slope 0.39 and memory slope 0.54: at n = 10,000 it took 5.96 ms and 151.4 KB. The test strings use at most 62 distinct characters, so the heap and tree stay small (cost is about O(n + k log k) for k distinct symbols) and the main n-dependent work is a

linear counting pass.

- **Measurement noise.** Timings below about 1 ms are noisy (for example Huffman is 1.85 ms at n = 2,500 against 1.76 ms at n = 1,000), so only the overall trend across sizes should be read from them.

7. Complexity Comparison Table

Table 3 compares the theoretical time and space complexity of each greedy algorithm with the values observed by the benchmarking engine (27 data points), using the largest input of each. The observed slope is the gradient of log(time) against log(n); for an O(n log n) algorithm it should be slightly above 1.

Algorithm	Theory time	Theory space	Input range	Time at max n (ms)	Ops at max n	Peak mem at max n (KB)	Observed slope (time)
Fractional Knapsack	O(n log n)	O(n)	10 - 10,000	140.2810	126,102	1,923.62	1.01 (theory ~1.1)
Activity Selection	O(n log n)	O(n)	10 - 10,000	60.1385	130,126	2,261.45	0.96 (theory ~1.1)
Huffman Coding	O(n log n)	O(n)	10 - 10,000	5.9603	20,787	151.36	0.39 (theory ~1, see note)

Table 3: Theoretical vs observed performance (source: benchmark_results.csv).

Algorithm	Growth trend observed
Fractional Knapsack	Near-linear: n grows 1,000x (10 to 10,000) and time grows about 1,178x. Operation count rises from 32 to 126,102 (n log n shape); memory slope 0.98.
Activity Selection	Near-linear: time grows about 700x for 1,000x input. Operations rise from 35 to 130,126 (the sort on finish time dominates); memory slope 1.00.
Huffman Coding	Sub-linear in n: time grows only about 20x. The heap work depends on the 62 distinct characters (saturated from n = 2,500), so only the linear text scan keeps growing; memory slope 0.54.

Notes: the space column is the theoretical auxiliary space; all three are O(n) because they build a sorted copy / selected list (Knapsack, Activity) or a tree and codebook (Huffman, where n is the number of distinct characters). An n log n curve fitted over 10 - 10,000 gives a slope a little above 1, but fixed per-call overhead at small n pulls the measured slope down to about 1. For Huffman the theoretical n is the number of distinct characters (capped at 62 here) while the benchmark varies the text length, so its measured slope is below 1. Each timing is a single run, so small deviations are expected.

8. Overall Comparison

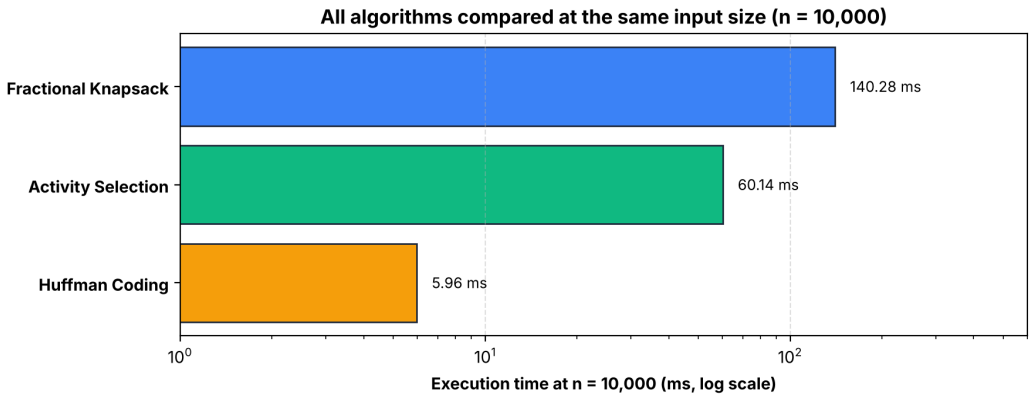


Figure 8: All three algorithms measured at the same input size (n = 10,000).

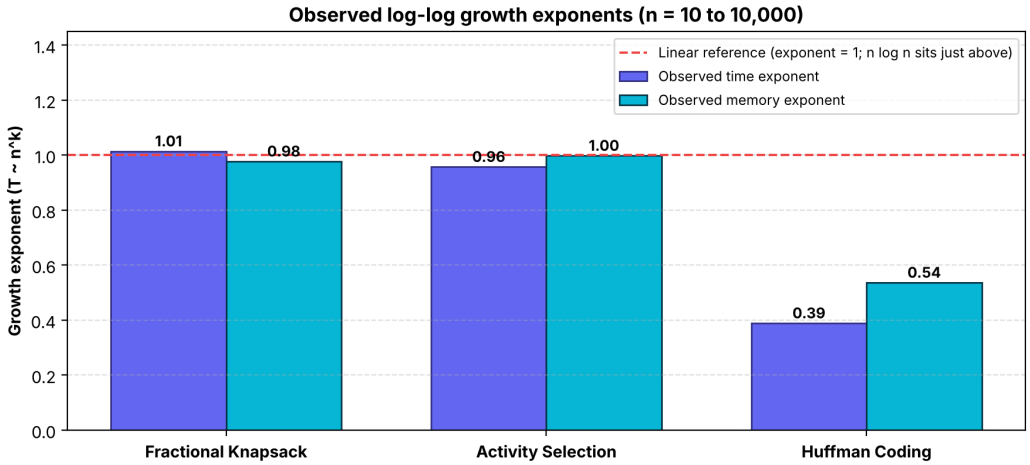


Figure 9: Observed log-log growth exponents for time and memory (n = 10 to 10,000).

Algorithm	Greedy criterion	Time	Space	Result in this lab
Fractional Knapsack	Highest v/w ratio	O(n log n)	O(n)	Max profit 285.00 (W = 50)
Activity Selection	Earliest finish time	O(n log n)	O(n)	3 of 8 activities scheduled
Huffman Coding	Merge two lowest frequencies	O(n log n)	O(n)	88 to 23 bits (73.9% saved)

Table 4: Complexity summary. For Huffman, n is the number of characters in the input; the heap work depends on the number of distinct symbols.

- At n = 10,000 the knapsack run (140.28 ms) is 2.3x slower than activity selection (60.14 ms) and 23.5x slower than Huffman (5.96 ms), even though all three are described as O(n log n). Constant factors, per-item object creation and, for Huffman, the small alphabet account for the gap.
- Knapsack is slower mainly because it creates a KnapsackItem object (with a ratio computation) for every item and sorts on a float key.
- Each of the three algorithms finishes 10,000 items in well under a quarter of a second and uses at most about 2.2 MB, so all are practical at this scale.
- Greedy is correct here because each problem has the greedy-choice property and optimal substructure. It does not carry over to 0/1 knapsack (W = 50, items 10/60, 20/100, 30/120: greedy gives 160, the optimum is 220), which needs dynamic programming.
- Real-world uses: fractional resource slicing in cloud computing (knapsack), room and CPU scheduling (activity selection), and entropy coding in JPEG, MP3 and ZIP (Huffman).

9. Discussion

- **Theory and experiment agree.** The benchmark data support O(n log n) time and O(n) space; no algorithm showed super-linear growth beyond the n log n trend.
- **Sorting dominates.** Knapsack and Activity Selection spend almost all their time in the sort; the greedy scan afterwards is linear.
- **Input structure matters.** Huffman's cost depends on the number of distinct symbols, not only on the text length, so its measured slope against text length is below 1.
- **Greedy is not always optimal.** It is optimal for fractional knapsack, activity selection (earliest finish) and Huffman coding, but fails for 0/1 knapsack.

10. Limitations

- **Single run per size:** each (algorithm, n) pair is timed once, so timings below about 1 ms are noisy.

- **Operation counts** come from a separate instrumented run (they are not stored in the CSV).
- **Memory** is the peak traced by tracemalloc during the call; the input is built before tracing starts.
- **Language and machine:** results come from CPython; absolute values will differ on another computer.
- **Huffman input:** the alphabet is capped at 62 characters, so its slope reflects the linear scan rather than the heap work.

11. Conclusion

The three greedy algorithms of Lab 3 - Fractional Knapsack, Activity Selection and Huffman Coding - were implemented, visualized and benchmarked for nine input sizes. Fractional Knapsack allocated the 50 kg capacity for a maximum profit of 285.00, Activity Selection scheduled 3 of 8 activities without overlap, and Huffman Coding compressed 'abracadabra' from 88 to 23 bits (73.9% saved). The benchmarks confirm O(n log n) time and O(n) space behaviour, and show that constant factors and input structure explain why algorithms with the same complexity class run at very different speeds.

Appendix A: Source Code

The three greedy algorithm modules and the benchmark runner. The menu-driven app (app.py), the Jupyter notebook and the plotting functions are in the repository and are not reproduced here.

algorithms/fractional_knapsack.py

```
"""
Resource Allocation Module: Fractional Knapsack Algorithm
Greedy Strategy: Value-to-Weight Ratio Maximization
Time Complexity: O(n log n)   Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""
from typing import List, Dict, Tuple, Union

class KnapsackItem:
    def __init__(self, name: str, weight: float, value: float):
        self.name = name
        self.weight = float(weight)
        self.value = float(value)
        self.ratio = self.value / self.weight if self.weight > 0 else 0.0

    def __repr__(self):
        return f"Item({self.name}, w={self.weight}, v={self.value}, ratio={self.ratio:.2f})"

def fractional_knapsack(items: List[Union[Dict, KnapsackItem]], capacity: float
                        ) -> Tuple[float, List[Tuple[str, float, float, float]]]:
    """Returns (total_value, allocation); allocation = (name, fraction, weight, value)."""
    if capacity <= 0 or not items:
        return 0.0, []

    parsed_items: List[KnapsackItem] = []
    for it in items:
        if isinstance(it, KnapsackItem):
            parsed_items.append(it)
        else:
            parsed_items.append(KnapsackItem(it["name"], it["weight"], it["value"]))

    # Greedy choice: sort in DESCENDING order of value-to-weight ratio
    sorted_items = sorted(parsed_items, key=lambda x: x.ratio, reverse=True)

    total_value = 0.0
    allocation = []
    remaining_capacity = float(capacity)

    for item in sorted_items:
        if remaining_capacity <= 0:
            break
        if item.weight <= remaining_capacity:
            # take the whole item
            total_value += item.value
            remaining_capacity -= item.weight
            allocation.append((item.name, 1.0, item.weight, item.value))
        else:
            # take a fraction of the item to fill the knapsack
            fraction = remaining_capacity / item.weight
            val_taken = item.value * fraction
            total_value += val_taken
            allocation.append((item.name, fraction, remaining_capacity, val_taken))
            remaining_capacity = 0.0
    return total_value, allocation

if __name__ == "__main__":
    test_items = [
        {"name": "Gold", "weight": 10, "value": 60},
        {"name": "Silver", "weight": 20, "value": 100},
        {"name": "Bronze", "weight": 30, "value": 120},
    ]
    max_val, details = fractional_knapsack(test_items, capacity=50)
    print(f"Maximum Achievable Profit: {max_val:.2f}")
    for name, frac, wt, val in details:
        print(f"  {name}: {frac*100:.1f}% taken | Weight: {wt:.1f} | Value: {val:.1f}")
```

algorithms/activity_selection.py

```
"""
Task Scheduling Module: Activity Selection Algorithm
Greedy Strategy: Earliest Finish Time First
Time Complexity: O(n log n)   Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""
from typing import List, Dict, Union

class Activity:
    def __init__(self, name: str, start: float, finish: float):
        self.name = name
        self.start = float(start)
        self.finish = float(finish)
        self.duration = self.finish - self.start

    def __repr__(self):
        return f"Activity({self.name}, [{self.start} -> {self.finish}], dur={self.duration})"

def activity_selection(activities: List[Union[Dict, Activity]]) -> List[Dict]:
    """Selects the maximum number of mutually compatible, non-overlapping activities."""
    if not activities:
        return []

    parsed_acts: List[Activity] = []
    for a in activities:
        if isinstance(a, Activity):
            parsed_acts.append(a)
        else:
            parsed_acts.append(Activity(a["name"], a["start"], a["finish"]))

    # Greedy choice: sort in ASCENDING order of FINISH times
    sorted_acts = sorted(parsed_acts, key=lambda a: a.finish)

    selected: List[Dict] = []
    first = sorted_acts[0]          # always take the earliest-finishing activity
    selected.append({"name": first.name, "start": first.start,
                     "finish": first.finish, "duration": first.duration})
    last_finish = first.finish

    for act in sorted_acts[1:]:
        if act.start >= last_finish:   # compatible with the last selected one
            selected.append({"name": act.name, "start": act.start,
                             "finish": act.finish, "duration": act.duration})
            last_finish = act.finish
    return selected

if __name__ == "__main__":
    sample = [
        {"name": "A1", "start": 1, "finish": 4}, {"name": "A2", "start": 3, "finish": 5},
        {"name": "A3", "start": 0, "finish": 6}, {"name": "A4", "start": 5, "finish": 7},
        {"name": "A5", "start": 3, "finish": 9}, {"name": "A6", "start": 5, "finish": 9},
        {"name": "A7", "start": 6, "finish": 10}, {"name": "A8", "start": 8, "finish": 11},
    ]
    scheduled = activity_selection(sample)
    print(f"Selected {len(scheduled)} Activities")
    for act in scheduled:
        print(f"  {act['name']}: Start={act['start']}, Finish={act['finish']}")
```

algorithms/huffman_coding.py

```
"""
Data Compression Module: Huffman Coding Algorithm
Greedy Strategy: Merge the two lowest-frequency nodes (prefix-free codes)
Time Complexity: O(n log n), n = distinct characters   Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""
import heapq
from collections import Counter
from typing import Dict, Tuple, Optional

class Node:
    """A node of the Huffman tree."""
    def __init__(self, char, freq, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):            # min-heap ordering by frequency
        return self.freq < other.freq

    def is_leaf(self):
        return self.left is None and self.right is None

def build_huffman_tree(text: str) -> Optional[Node]:
```

```
    if not text:
        return None
    frequency = Counter(text)
    heap = [Node(char=ch, freq=f) for ch, f in frequency.items()]
    heapq.heapify(heap)
    if len(heap) == 1:                  # only one distinct character
        single = heapq.heappop(heap)
        return Node(char=None, freq=single.freq, left=single, right=None)
    while len(heap) > 1:                # greedily merge two lowest frequencies
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        heapq.heappush(heap, Node(None, left.freq + right.freq, left, right))
    return heap[0]

def generate_codes(node, prefix="", codebook=None) -> Dict[str, str]:
    if codebook is None:
        codebook = {}
    if node is None:
        return codebook
    if node.is_leaf():
        codebook[node.char] = prefix if prefix else "0"
        return codebook
    if node.left:
        generate_codes(node.left, prefix + "0", codebook)
    if node.right:
        generate_codes(node.right, prefix + "1", codebook)
    return codebook

def huffman_encode(text: str):
    """Returns (encoded_bits, codebook, frequencies, stats)."""
    if not text:
        return "", {}, {}, {"original_bits": 0, "compressed_bits": 0, "ratio": 0.0, "savings": 0.0}
    root = build_huffman_tree(text)
    codebook = generate_codes(root)
    frequencies = dict(Counter(text))
    encoded_bits = "".join(codebook[ch] for ch in text)

    original_bits = len(text) * 8                    # 8-bit ASCII
    compressed_bits = len(encoded_bits)
    ratio = compressed_bits / original_bits
    stats = {
        "original_chars": len(text), "distinct_chars": len(frequencies),
        "original_bits": original_bits, "compressed_bits": compressed_bits,
        "compression_ratio": ratio, "space_savings_percent": (1.0 - ratio) * 100.0,
        "avg_code_length": compressed_bits / len(text),
    }
    return encoded_bits, codebook, frequencies, stats

def huffman_decode(encoded_bits: str, root: Node) -> str:
    if not encoded_bits or not root:
        return ""
    out, curr = [], root
    for bit in encoded_bits:
        curr = (curr.left or curr) if bit == "0" else (curr.right or curr)
        if curr.is_leaf():
            out.append(curr.char)
            curr = root
    return "".join(out)

if __name__ == "__main__":
    text = "abracadabra"
    bits, codes, freqs, st = huffman_encode(text)
    print("Codebook:", codes, "| Encoded:", bits)
    print(f"Original: {st['original_bits']} bits | Compressed: {st['compressed_bits']} bits")
    assert huffman_decode(bits, build_huffman_tree(text)) == text
    print("Lossless decompression verified successfully!")
```

analysis/generate_benchmarks_and_charts.py (run_benchmarks)

```
def run_benchmarks():
    print("[*] Running comprehensive benchmark experiments for all 3 greedy algorithms...")
    input_sizes = [10, 50, 100, 250, 500, 1000, 2500, 5000, 10000]
    results = []

    for n in input_sizes:
        print(f"  Benchmarking size n = {n} ...")

        # 1. Fractional Knapsack
        knapsack_items = [
            {"name": f"Item_{i}", "weight": random.uniform(5, 100), "value": random.uniform(10, 500)}
            for i in range(n)
        ]
        cap = n * 25.0
        tracemalloc.start()
        t0 = time.perf_counter()
        fractional_knapsack(knapsack_items, capacity=cap)
        t_knapsack = time.perf_counter() - t0
        _, mem_knapsack = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        # 2. Activity Selection
        activities = []
        for i in range(n):
            s = random.randint(0, n * 2)
            f = s + random.randint(1, 50)
            activities.append({"name": f"A_{i}", "start": s, "finish": f})
        tracemalloc.start()
        t0 = time.perf_counter()
        activity_selection(activities)
        t_activity = time.perf_counter() - t0
        _, mem_activity = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        # 3. Huffman Coding (text with skewed character frequencies)
        charset = string.ascii_letters + string.digits
        weights = [1.0 / (i + 1) for i in range(len(charset))]
        text_sample = "".join(random.choices(charset, weights=weights, k=n))
        tracemalloc.start()
        t0 = time.perf_counter()
        huffman_encode(text_sample)
        t_huffman = time.perf_counter() - t0
        _, mem_huffman = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        results.append({
            "Input_Size_n": n,
            "Knapsack_Time_sec": t_knapsack, "Knapsack_Mem_KB": mem_knapsack / 1024.0,
            "Activity_Time_sec": t_activity, "Activity_Mem_KB": mem_activity / 1024.0,
            "Huffman_Time_sec": t_huffman,   "Huffman_Mem_KB": mem_huffman / 1024.0,
        })

    df = pd.DataFrame(results)
    csv_path = os.path.join(DATA_DIR, "benchmark_results.csv")
    df.to_csv(csv_path, index=False)
    print(f"[OK] Benchmark results saved to {csv_path}")
    return df
```

Appendix B: Complete Measurement Data

All nine records of benchmark_results.csv (time in ms, memory in KB). Operations at n = 10,000 and the theoretical complexity are given in Table 3.

n	Knapsack time (ms)	Knapsack mem (KB)	Activity time (ms)	Activity mem (KB)	Huffman time (ms)	Huffman mem (KB)
10	0.1191	2.75	0.0859	2.48	0.3043	2.92
50	0.3578	7.41	0.3858	10.68	0.5298	8.00
100	0.6755	14.35	0.7214	20.27	0.7476	10.97
250	2.4506	41.91	1.9168	54.14	1.1844	19.46
500	3.7750	82.68	4.0090	110.04	1.4236	23.88
1,000	8.0878	166.93	8.2191	223.14	1.7599	31.75
2,500	15.2443	431.63	13.9410	564.02	1.8480	50.90
5,000	48.2021	836.10	36.2427	1126.12	2.9401	84.16
10,000	140.2810	1923.62	60.1385	2261.45	5.9603	151.36